深度学习基础

第 8 讲 — 循环神经网络与序列建模

孟翰

人工智能学院
中国石油大学（北京）

2026 年春季学期

本讲内容

1. 回顾与本讲目标
2. 序列数据与建模任务
3. RNN：循环神经网络
4. BPTT 与梯度问题
5. LSTM 与 GRU：门控机制
6. 文本处理与 PyTorch 实现
7. 小结与下讲预告

CNN 的核心思想

先验决定结构：图像有 “局部性 + 平移不变性”→ 卷积；CNN 是**为图像量身定制**的网络。

下一个问题：序列数据呢?

文本、语音、股价、传感器读数……这些数据有什么不同?

- **有时间顺序**：早晚不能颠倒
- **长度可变**：一句话可能 5 个词也可能 50 个词
- **元素之间有依赖**：当前依赖于过去

CNN 和 MLP 都不擅长处理序列

- MLP：固定输入维度，无法处理变长
- CNN：可以处理 1D 序列，但感受野有限

我们需要一种 “有记忆” 的网络。

一句话目标

理解**循环神经网络（RNN）**如何用“隐状态”建模序列，以及为什么需要 LSTM/GRU 这样的**门控变体**。

学完本讲，你能

- 解释 RNN 的隐状态机制
- 推导 BPTT，理解梯度消失/爆炸的根源
- 说出 LSTM 三个门各自的作用
- 用 PyTorch 搭建 RNN/GRU 做文本分类

重要说明

RNN 在 2025 年已不是序列建模的主流（被 Transformer 取代）。但理解 RNN 是理解 Transformer 的最佳前提，且 RNN 的“隐状态”思想在 Mamba 等新模型中重新焕发生机。

形式化定义

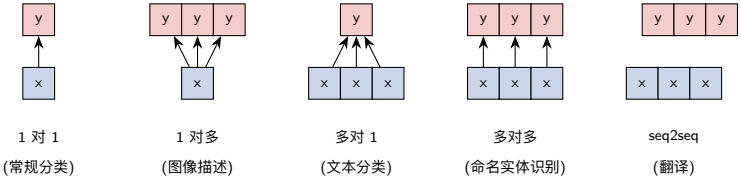
一个序列：$\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_T$，$T$ 是序列长度。每个元素 $\boldsymbol{x}_t$ 是一个向量。

常见序列数据

类型	元素	典型任务
文本	词/字符	翻译、分类、生成
语音	帧 (MFCC 等)	识别、合成
时间序列	数值	预测、异常检测
视频	帧图像	动作识别
EEG/ECG	通道值	疾病诊断

核心特征

顺序信息至关重要。“我爱你”和“你爱我”用相同的字，意思不同。



核心观察

不同任务对模型的要求不同。RNN 通过**隐状态**的不同使用方式，可以同时支持这五种范式。

MLP 的结构性限制

多层感知机要求输入维度固定。若将长度为 T 的序列拼接为一个长向量 $\boldsymbol{x} \in \mathbb{R}^{Td}$ 输入 MLP，将面临三个问题：(1) 参数量随 T 线性增长；(2) 每个时间步对应独立的权重，无法共享；(3) 序列长度一旦改变，整个网络必须重新设计与训练。

一维 CNN 的局限

一维卷积虽可处理序列，但其感受野由卷积核大小与网络深度共同决定。欲捕捉跨越数十乃至上百步的依赖关系，需堆叠大量层，既增加计算成本，也加剧优化困难。

目标：具有 “记忆” 能力的网络

理想的序列模型应满足：(a) 参数量与序列长度 T 无关；(b) 能够在不同时间步间共享参数；(c) 天然支持任意长度的输入。RNN 通过引入**隐状态**与**参数共享**机制同时满足上述三项要求。

基本想法

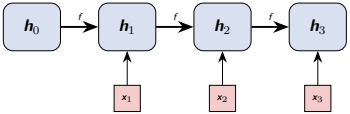
让网络维护一个“记忆”（隐状态 $\boldsymbol{h}_t$），它**随时间更新**：$\boldsymbol{h}_t = f(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$

直观解释

当前隐状态等于函数 f 作用于上一时刻隐状态与当前输入的结果。其机理可类比于阅读过程：读者对当前句子的理解同时依赖于句子本身的词句，以及之前已经读到的上下文内容。隐状态 $\boldsymbol{h}_t$ 即承担了“已读上下文”这一角色。

关键性质

同一个 f（同一组参数）在所有时间步**重复使用**。**参数共享**是 RNN 节省参数的关键。



标准 RNN（Vanilla RNN）

$$\boxed{\boldsymbol{h}_t = \tanh(\boldsymbol{W}_{hh}\,\boldsymbol{h}_{t-1} + \boldsymbol{W}_{xh}\,\boldsymbol{x}_t + \boldsymbol{b}_h)}$$

$$\boldsymbol{y}_t = \boldsymbol{W}_{hy}\,\boldsymbol{h}_t + \boldsymbol{b}_y$$

参数清单（与 T 无关！）

- $\boldsymbol{W}_{xh}$：输入 $\to$ 隐藏的权重，$\mathbb{R}^{H\times d}$
- $\boldsymbol{W}_{hh}$：隐藏 $\to$ 隐藏的权重，$\mathbb{R}^{H\times H}$
- $\boldsymbol{W}_{hy}$：隐藏 $\to$ 输出的权重，$\mathbb{R}^{c\times H}$
- $\boldsymbol{b}_h, \boldsymbol{b}_y$：偏置

重要

无论序列多长，参数数量不变。这就是 RNN 能处理变长序列的原因。

设定

设输入维度 $d = 3$，隐藏维度 $H = 4$，输出类别数 $c = 2$，序列长度 $T = 5$，批大小 $B = 1$。

每步计算流程

1. 初始化 $\boldsymbol{h}_0 = \mathbf{0} \in \mathbb{R}^4$（常见做法为零向量，亦可作为可学习参数）
2. 对 $t = 1, 2, \ldots, 5$，依次执行：
   - 读取输入 $\boldsymbol{x}_t \in \mathbb{R}^3$
   - 计算线性变换 $\boldsymbol{z}_t = \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{b}_h \in \mathbb{R}^4$
   - 施加非线性 $\boldsymbol{h}_t = \tanh(\boldsymbol{z}_t) \in \mathbb{R}^4$
   - 计算输出 $\boldsymbol{y}_t = \boldsymbol{W}_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y \in \mathbb{R}^2$
3. 根据任务类型收集所需输出：多对一任务仅取 $\boldsymbol{y}_5$；多对多任务收集全部 $\{\boldsymbol{y}_t\}_{t=1}^5$。

参数总量计算

$|\boldsymbol{W}_{xh}| = H \times d = 12$，$|\boldsymbol{W}_{hh}| = H^2 = 16$，$|\boldsymbol{W}_{hy}| = c \times H = 8$，加偏置 $4 + 2 = 6$，合计 **42 个参数**，与 T 无关。

核心问题

在每个时间步 t，RNN 单元需要同时“消化”两股信息：(1) 来自上一时刻的隐状态 $\boldsymbol{h}_{t-1} \in \mathbb{R}^H$；(2) 当前时刻的输入 $\boldsymbol{x}_t \in \mathbb{R}^d$。这两个向量维度不同、语义也不同，必须通过某种方式**融合为单一向量**后再经非线性激活。

两种等价的融合表达

形式一（分离矩阵）：

$$\boldsymbol{h}_t = \tanh\big(\boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{b}_h\big)$$

形式二（拼接矩阵）： 将 $\boldsymbol{h}_{t-1}$ 与 $\boldsymbol{x}_t$ 沿特征维拼接为 $[\boldsymbol{h}_{t-1};\, \boldsymbol{x}_t] \in \mathbb{R}^{H+d}$，再用单一权重 $\boldsymbol{W} \in \mathbb{R}^{H\times(H+d)}$ 做一次线性变换：

$$\boldsymbol{h}_t = \tanh\big(\boldsymbol{W}[\boldsymbol{h}_{t-1};\, \boldsymbol{x}_t] + \boldsymbol{b}_h\big)$$

两种形式数学等价

将分块矩阵 $\boldsymbol{W} = [\boldsymbol{W}_{hh} \mid \boldsymbol{W}_{xh}]$ 代入形式二即可还原形式一。PyTorch 在底层实现时通常采用**形式二**以减少矩阵乘法的次数，提高 GPU 利用率。LSTM/GRU 公式中频繁出现的 $[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t]$ 即为此种拼接。

设定：$d = 2, H = 3$ 的微型 RNN

令 $\boldsymbol{x}_t = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$，$\boldsymbol{h}_{t-1} = \begin{pmatrix} h_1 \\ h_2 \\ h_3 \end{pmatrix}$。拼接后 $[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] = (h_1, h_2, h_3, x_1, x_2)^\top \in \mathbb{R}^5$。

分块权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{3\times 5}$

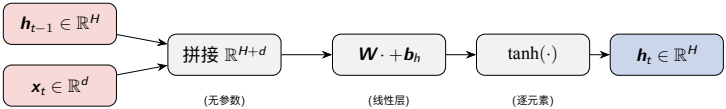
$$\boldsymbol{W} = \begin{pmatrix} \underbrace{w_{11} \;\; w_{12} \;\; w_{13}}_{\boldsymbol{W}_{hh}\text{ 第 1 行}} & \underbrace{w_{14} \;\; w_{15}}_{\boldsymbol{W}_{xh}\text{ 第 1 行}} \\ w_{21} \;\; w_{22} \;\; w_{23} & w_{24} \;\; w_{25} \\ w_{31} \;\; w_{32} \;\; w_{33} & w_{34} \;\; w_{35} \end{pmatrix}$$

$\boldsymbol{W}$ **的左三列**负责“读取历史”（对应 $\boldsymbol{h}_{t-1}$），**右两列**负责“读取当前输入”（对应 $\boldsymbol{x}_t$）。

预激活向量 $\boldsymbol{z}_t \in \mathbb{R}^3$ 的显式计算

$$\boldsymbol{z}_t^{(1)} = \underbrace{w_{11}h_1 + w_{12}h_2 + w_{13}h_3}_{\text{历史贡献}} + \underbrace{w_{14}x_1 + w_{15}x_2}_{\text{输入贡献}} + b_1$$

即 $\boldsymbol{z}_t$ 的每一维都是“历史信息加权和”与“当前输入加权和”的**相加融合**，再加偏置。

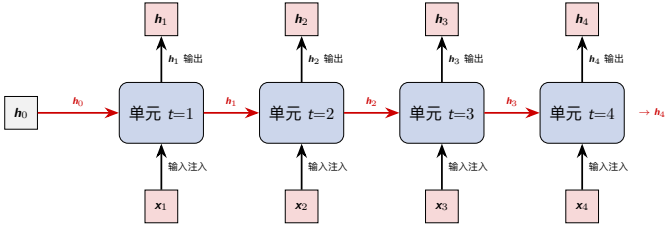


五步流水线（单时间步内部）

1. **拼接**：$\boldsymbol{h}_{t-1}$ 与 $\boldsymbol{x}_t$ 首尾相接，无可学习参数，不改变数值
2. **线性变换**：$\boldsymbol{z}_t = \boldsymbol{W} \cdot [\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_h$，一次矩阵向量乘法
3. **非线性激活**：$\boldsymbol{h}_t = \tanh(\boldsymbol{z}_t)$，逐元素作用，将数值压缩至 $(-1, 1)$
4. **状态传递**：$\boldsymbol{h}_t$ 沿时间方向送入下一时刻，同时作为输出层的输入
5. **输出生成**（可选）：$\boldsymbol{y}_t = \boldsymbol{W}_{hy}\boldsymbol{h}_t + \boldsymbol{b}_y$

关键观察

整个流水线**没有随时间变化的参数**。同一套 $\boldsymbol{W}, \boldsymbol{b}_h$ 在 $t = 1, 2, \ldots, T$ 所有步骤中重复使用。改变的只是流经的数据。



三条信息流通道

- **纵向向下**（红色输入）：每个时间步独立注入新的 $\boldsymbol{x}_t$
- **横向水平**（红色箭头）：隐状态 $\boldsymbol{h}_t$ 承载“历史摘要”在时间轴上传递
- **纵向向上**：每步产出 $\boldsymbol{h}_t$ 供输出层或后续任务使用

信息瓶颈的由来

无论序列多长，历史信息在每一步都被**压缩到固定维度** H 的向量中传递。当 T 远大于 H 时，$\boldsymbol{h}_t$ 必然丢失早期的细节信息。这正是 RNN 的根本局限之一，也是后续 Transformer 注意力机制要解决的核心问题。

引入批维度

训练时通常同时处理 B 个序列。此时输入张量形状为 (B, T, d)，隐状态为 (B, H)。所有矩阵乘法都**在批维度上并行**执行。

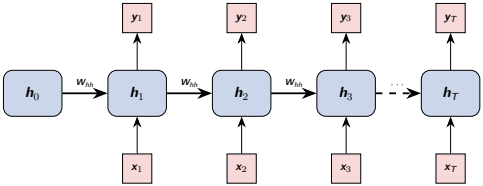
单个时间步的批处理计算

设 $B = 4, d = 2, H = 3$。

- 取出该时间步所有样本的输入 $\boldsymbol{X}_t \in \mathbb{R}^{4\times 2}$
- 取出上一时刻所有样本的隐状态 $\boldsymbol{H}_{t-1} \in \mathbb{R}^{4\times 3}$
- 沿特征维拼接：$[\boldsymbol{H}_{t-1}, \boldsymbol{X}_t] \in \mathbb{R}^{4\times 5}$
- 计算 $\boldsymbol{Z}_t = [\boldsymbol{H}_{t-1}, \boldsymbol{X}_t]\boldsymbol{W}^\top + \boldsymbol{b}_h \in \mathbb{R}^{4\times 3}$（广播加法）
- 激活 $\boldsymbol{H}_t = \tanh(\boldsymbol{Z}_t) \in \mathbb{R}^{4\times 3}$

并行的维度与串行的维度

批维度 B 上**完全并行**（GPU 一次性处理所有样本）；但时间维度 T 上**严格串行**——必须先算完 $\boldsymbol{H}_1$ 才能算 $\boldsymbol{H}_2$。这种时间维的串行性正是 RNN 无法充分利用 GPU 并行能力的根本原因，也是相较 Transformer 的主要劣势。



重要观察

- 这**不是** T **个不同的网络**，而是**同一个网络重复使用** T **次**
- **所有时间步共享同一个** $\boldsymbol{W}_{xh}, \boldsymbol{W}_{hh}$
- 展开后看起来像一个深度为 T 的前馈网络

核心思想

RNN 展开后是**深度前馈网络**（深度 $= T$）。**反向传播照样适用**，只不过梯度需要 “沿时间倒流”。这就是 BPTT（Back-Propagation Through Time）。

实际操作

1. **前向**：从 $t = 1$ 到 T 计算所有 $\boldsymbol{h}_t$, $\boldsymbol{y}_t$, 损失 $\mathcal{L}_t$
2. **总损失**：$\mathcal{L} = \sum_t \mathcal{L}_t$
3. **反向**：从 $t = T$ 倒推回 $t = 1$，累积梯度
4. **更新**：用累积梯度更新共享参数 $\boldsymbol{W}_{hh}, \boldsymbol{W}_{xh}$

严重问题

当 T 很大（如 $T = 100$）时，梯度需要经过 **100 次连乘**。这会带来**梯度消失或爆炸**。

损失对 $\boldsymbol{W}_{hh}$ 的梯度

以 $T = 3$、损失 $\mathcal{L} = \mathcal{L}_3$ 为例。由链式法则：

$$\frac{\partial \mathcal{L}_3}{\partial \boldsymbol{W}_{hh}} = \sum_{k=1}^{3} \frac{\partial \mathcal{L}_3}{\partial \boldsymbol{h}_3} \cdot \frac{\partial \boldsymbol{h}_3}{\partial \boldsymbol{h}_k} \cdot \frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{W}_{hh}}$$

关键的连乘项

由 $\boldsymbol{h}_t = \tanh(\boldsymbol{W}_{hh}\boldsymbol{h}_{t-1} + \boldsymbol{W}_{xh}\boldsymbol{x}_t + \boldsymbol{b}_h)$ 可得：

$$\frac{\partial \boldsymbol{h}_3}{\partial \boldsymbol{h}_1} = \frac{\partial \boldsymbol{h}_3}{\partial \boldsymbol{h}_2} \cdot \frac{\partial \boldsymbol{h}_2}{\partial \boldsymbol{h}_1} = \left[\boldsymbol{W}_{hh}^\top \mathrm{diag}(\tanh')\right]_{t=3} \cdot \left[\boldsymbol{W}_{hh}^\top \mathrm{diag}(\tanh')\right]_{t=2}$$

本质观察

由于参数 $\boldsymbol{W}_{hh}$ 在所有时间步共享，同一个矩阵在反向传播中被**反复相乘**。当 T 增大时，连乘项的数量与 T 成正比，这正是梯度消失与梯度爆炸的直接根源。

反向传播链式法则

对 $\boldsymbol{W}_{hh}$ 求梯度时会出现：

$$\frac{\partial \boldsymbol{h}_t}{\partial \boldsymbol{h}_k} = \prod_{i=k+1}^{t} \boldsymbol{W}_{hh}^{\top} \cdot \mathrm{diag}(\tanh')$$

两种情况

- 若 $\|\boldsymbol{W}_{hh}\| < 1$：连乘 $\rightarrow$ 指数衰减 $\rightarrow$ **梯度消失**
- 若 $\|\boldsymbol{W}_{hh}\| > 1$：连乘 $\rightarrow$ 指数增长 $\rightarrow$ **梯度爆炸**

后果

梯度消失：网络学不到长距离依赖（如 “今天天气取决于 30 天前的某事件”）。**梯度爆炸**：训练发散，损失变 NaN。

数值实验

假设 $\|\boldsymbol{W}_{hh}^{\top}\mathrm{diag}(\tanh')\| \approx 0.9$（这是相当温和的情形）。

梯度衰减速度

时间跨度 $t-k$	10	20	50	100	200	500
0.9^{t-k}	0.35	0.12	5.2×10^{-3}	2.7×10^{-5}	7.1×10^{-10}	≈ 0

实际意义

即便系数仅略小于 1，经过 100 步的连乘后，早期时间步对当前损失的梯度贡献已衰减至 10^{-5} 量级，在数值上几乎无法参与参数更新。这意味着网络**无法从远距离的监督信号中学习**。

对称地

若谱范数略大于 1（例如 1.1），则 $1.1^{100} \approx 1.38\times10^{4}$，梯度将迅速膨胀，导致训练发散。

爆炸：梯度裁剪（Gradient Clipping）

设阈值 θ，若 $\|\nabla\| > \theta$，则 $\nabla \leftarrow \theta \cdot \nabla / \|\nabla\|$。
该方法实现简洁且行之有效，已成为 RNN 训练流程中的标准配置。

消失：架构层面的修改

不能靠裁剪解决。需要一种**结构上能保留长距离信息**的网络：

- **LSTM**（1997）：用 “细胞状态” + 三个门
- **GRU**（2014）：简化版 LSTM，两个门

核心思想（与 ResNet 异曲同工）

建立一条 “梯度直通车”，让信息可以**无损地**通过多个时间步。

核心创新（Hochreiter & Schmidhuber, 1997）

引入**细胞状态** $\boldsymbol{c}_t$（长期记忆），与隐状态 $\boldsymbol{h}_t$（短期记忆）分开。
用三个**门**控制信息流动。

三个门（每个门都是 sigmoid，输出 [0,1]）

- **遗忘门** $\boldsymbol{f}_t$：决定从 $\boldsymbol{c}_{t-1}$ 中**丢弃**什么
- **输入门** $\boldsymbol{i}_t$：决定向 $\boldsymbol{c}_t$ 中**写入**什么
- **输出门** $\boldsymbol{o}_t$：决定从 $\boldsymbol{c}_t$ 中**读出**什么作为 $\boldsymbol{h}_t$

为什么有效

门是**加性更新**：$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t$。
当 $\boldsymbol{f}_t \approx 1$ 时，旧信息几乎无损保留。**这就是“梯度直通车”**。

门的数学本质

每个门都由一个 sigmoid 单元产生，输出为与 $\boldsymbol{c}_t$ 同维度的向量，各分量取值位于 $[0,1]$ 区间。门与目标向量做**逐元素乘法**（Hadamard 积 $\odot$），取值接近 0 表示“关闭”对应维度，取值接近 1 表示“放行”对应维度。因此门可视为**可学习的、动态的开关**。

以阅读任务为例

设模型正在处理句子“小明昨天去了北京，今天他……”。

- 读到“北京”时，**输入门**打开，将“地点 = 北京”写入 $\boldsymbol{c}_t$
- 读到句号时，**遗忘门**对部分与地点相关的维度输出较小值，逐步淡化该信息
- 读到“他”时，**输出门**控制是否将“主语 = 小明”从 $\boldsymbol{c}_t$ 读出以协助代词指代消解

上述流程中“写入—保留—读出”的职能分离，正是 LSTM 优于 vanilla RNN 的根本原因。

梯度直通车机制

$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t$ 是**加性**结构。当遗忘门 $\boldsymbol{f}_t \approx \boldsymbol{1}$ 时，$\partial \boldsymbol{c}_t / \partial \boldsymbol{c}_{t-1} \approx \boldsymbol{I}$，梯度沿细胞状态通道近乎无损地回传，从而有效缓解梯度消失问题。

完整公式

$$
\begin{aligned}
\boldsymbol{f}_t &= \sigma(\boldsymbol{W}_f[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_f) && \text{遗忘门} \\
\boldsymbol{i}_t &= \sigma(\boldsymbol{W}_i[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_i) && \text{输入门} \\
\tilde{\boldsymbol{c}}_t &= \tanh(\boldsymbol{W}_c[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_c) && \text{候选记忆} \\
\boldsymbol{c}_t &= \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t && \text{更新细胞} \\
\boldsymbol{o}_t &= \sigma(\boldsymbol{W}_o[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t] + \boldsymbol{b}_o) && \text{输出门} \\
\boldsymbol{h}_t &= \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t) && \text{隐状态}
\end{aligned}
$$

记住这一点

LSTM 的参数量大约是 RNN 的 **4 倍**（4 个权重矩阵），计算也慢约 4 倍。但在长序列上效果远超 vanilla RNN。

公式中重复出现的拼接向量

观察 LSTM 公式：四个门（$\boldsymbol{f}_t, \boldsymbol{i}_t, \boldsymbol{o}_t$）与候选状态 $\tilde{\boldsymbol{c}}_t$ 的输入**完全相同**，均为 $[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] \in \mathbb{R}^{H+d}$，仅权重矩阵不同。

单次矩阵乘法计算四个输出

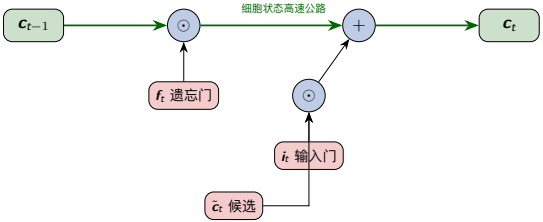
将四个权重矩阵沿行方向堆叠为单一大矩阵 $\boldsymbol{W}_{\text{all}} \in \mathbb{R}^{4H\times(H+d)}$：

$$\boldsymbol{W}_{\text{all}} = \begin{pmatrix} \boldsymbol{W}_f \\ \boldsymbol{W}_i \\ \boldsymbol{W}_o \\ \boldsymbol{W}_c \end{pmatrix}, \qquad \begin{pmatrix} \boldsymbol{z}_f \\ \boldsymbol{z}_i \\ \boldsymbol{z}_o \\ \boldsymbol{z}_c \end{pmatrix} = \boldsymbol{W}_{\text{all}}[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_{\text{all}}$$

随后将输出向量按维度 H 切成四段，前三段通过 σ 得到三个门，第四段通过 tanh 得到候选状态。

为何这样做

单次大矩阵乘法在 GPU 上的计算效率远高于四次小矩阵乘法。PyTorch 的 `nn.LSTM` 内部即采用此种融合实现。这解释了为何打印 LSTM 参数时会看到形状为 $(4H, H+d)$ 的权重张量。



数据流语义解读

细胞状态沿图中**绿色水平路径**从 c_{t-1} 直接传递到 c_t，途中仅经历两次简单操作：先与遗忘门逐元素相乘（部分清零），再加上由输入门控制的新内容。这条路径**不经过任何非线性挤压**（tanh 只作用于候选），因此梯度在回传时几乎不会被压缩。

对比 vanilla RNN

vanilla RNN 中 $\boldsymbol{h}_t = \tanh(\boldsymbol{W}\cdot)$，每一步都经过 tanh 的非线性压缩，其导数最大仅为 1，多步连乘必然衰减。LSTM 的细胞状态通道绕开了这种连乘结构。

GRU（Cho et al., 2014）

合并细胞状态和隐状态。只用两个门：

- **重置门** $\boldsymbol{r}_t$：决定多少历史信息进入候选状态
- **更新门** $\boldsymbol{z}_t$：决定多少历史保留、多少新信息覆盖

优点

- 参数比 LSTM 少 25%
- 训练更快
- 短/中序列效果与 LSTM 相当

GRU 公式

$$\boldsymbol{r}_t = \sigma(\boldsymbol{W}_r[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t])$$
$$\boldsymbol{z}_t = \sigma(\boldsymbol{W}_z[\boldsymbol{h}_{t-1}, \boldsymbol{x}_t])$$
$$\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}[\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}, \boldsymbol{x}_t])$$
$$\boldsymbol{h}_t = (1-\boldsymbol{z}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{z}_t \odot \tilde{\boldsymbol{h}}_t$$

选择建议

长序列（$T > 200$）：LSTM
短/中序列：GRU
效果差距通常很小

核心问题

RNN 需要的输入是**向量**，但文本是**字符/词**。怎么办？

两步走

1. **建词表**：把每个词映射到一个整数 ID（cat $\rightarrow$ 5）
2. **词嵌入**：用一个 $\mathbb{R}^{V\times d}$ 的查找表，把 ID 映射到 d 维向量

Embedding 层（PyTorch 里就一行）

```
nn.Embedding(num_embeddings=V, embedding_dim=128)
```

这就是词嵌入。词向量本身是**可学习的参数**，会随网络一起训练。

one-hot 编码的局限

最简单的离散表示是 one-hot：词表大小为 V 时，每个词用一个 V 维向量表示，仅第 ID 位为 1、其余为 0。存在两项严重缺陷：

- **维度灾难**：典型词表 $V = 10^4 \sim 10^6$，导致输入向量极度稀疏且存储代价高
- **语义缺失**：任意两个不同词的 one-hot 向量相互正交，余弦相似度恒为 0。“猫” 与 “狗” 的距离和 “猫” 与 “桌子” 的距离相同，无法反映语义相关性

词嵌入的改进

词嵌入将每个词映射到低维稠密向量（典型维度 $d = 128 \sim 512$）。由于嵌入矩阵参与梯度更新，**语义相近的词会在训练后自然聚集于相邻位置**。其几何结构甚至能够编码类比关系，如 $\mathrm{vec}(\text{国王}) - \mathrm{vec}(\text{男}) + \mathrm{vec}(\text{女}) \approx \mathrm{vec}(\text{女王})$。

实现要点

`nn.Embedding` 在底层实际上是一个 $V \times d$ 的查找表。前向传播时按整数 ID 索引取出对应行；反向传播时仅更新被访问过的行。

```python
import torch.nn as nn

class GRUClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden=256, num_classes=2):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim)
        self.gru = nn.GRU(input_size=embed_dim, hidden_size=hidden,
                          num_layers=2, batch_first=True, dropout=0.3)
        self.fc = nn.Linear(hidden, num_classes)

    def forward(self, x):
        # x shape: (batch, seq_len)
        emb = self.embed(x)              # (batch, seq_len, embed_dim)
        out, h_n = self.gru(emb)         # out: (batch, seq_len, hidden)
                                          # h_n: (num_layers, batch, hidden)
        last_h = h_n[-1]                 # 取最后一层最后时刻 (batch, hidden)
        return self.fc(last_h)           # (batch, num_classes)
```

对照第 6/7 讲

代码风格和 CNN 完全一样：`nn.Module` + `forward`。**只是把 Conv 换成了 GRU。**

以批大小 $B = 32$、序列长度 $T = 100$、词表 $V = 10000$ 为例

- **输入** `x`：形状 $(32, 100)$，元素为词表 ID（整数）
- **经 Embedding 后** `emb`：形状 $(32, 100, 128)$，每个 ID 被替换为 128 维向量
- **经 GRU 后**：
  - `out`：形状 $(32, 100, 256)$，记录所有时间步的隐状态
  - `h_n`：形状 $(2, 32, 256)$，其中 2 代表 GRU 的层数
- **取最后一层隐状态** `h_n[-1]`：形状 $(32, 256)$
- **经全连接层**：形状 $(32, 2)$，即每个样本的 2 类 logits

两种聚合策略

多对一任务中将变长序列压缩为单一向量的常见做法：(1) 取最后时刻隐状态 `h_n[-1]`；(2) 对所有时间步的 `out` 做时间维度上的均值池化或最大池化。后者在序列存在噪声或关键信息不位于末尾时通常更鲁棒。

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = GRUClassifier(vocab_size=10000).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(5):
    model.train()
    for x, y in train_loader:                    # x: (B, T) y: (B,)
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss = criterion(logits, y)
        optimizer.zero_grad()
        loss.backward()
        # 梯度裁剪（必须！）
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
        optimizer.step()
```

两个 RNN 训练的“必备技巧”

1. **梯度裁剪** `clip_grad_norm_`：防止梯度爆炸
2. `batch_first=True`：把 batch 维放在最前，符合常规习惯

Bug 1: 形状错误

RNN 输入要求形状 (`batch, seq_len, input_size`) (`batch_first=True`)。
常见错误：忘记设 `batch_first`，导致形状变成 (`seq_len, batch, input_size`)。

Bug 2: 序列长度不一致

一个 batch 内不同样本长度不同，需要 **padding** 到相同长度，并用 `pack_padded_sequence` 让 RNN 跳过 padding 部分。

训练技巧

- 用 **GRU 而非 RNN** 作为默认（除非数据极少）
- 学习率可以小一些（10^{-3} 或 5×10^{-4}）
- Dropout 加在 **层间**（不是时间步间），PyTorch 的 GRU/LSTM 自带
- 序列长 ≥ 200：考虑双向（`bidirectional=True`）

局限 1：无法并行

RNN 必须**按时间顺序**计算 $\boldsymbol{h}_1 \to \boldsymbol{h}_2 \to \boldsymbol{h}_3 \to \cdots$。**GPU 的并行能力浪费殆尽。**

局限 2：长距离依赖依然困难

即使有 LSTM，超过 100 步的依赖依然学不好。**信息需要“逐跳”传递。**

局限 3：固定的信息瓶颈

所有历史都被压缩到一个固定大小的 $\boldsymbol{h}_t$ 中。复杂的句子会丢信息。

下讲：注意力机制

Transformer 用“注意力”一举解决这三个问题：可以并行、可以直连任意位置、信息瓶颈消失。

2017 年的“Attention is All You Need”彻底改变了序列建模。

三个关键概念

1. **隐状态 h_t**：让网络拥有“记忆”，是 RNN 的核心
2. **BPTT**：通过时间的反向传播，会带来梯度消失/爆炸
3. **门控机制**（LSTM/GRU）：用加性更新打通“梯度直通车”

两个工程技巧（必记）

- **梯度裁剪** `clip_grad_norm_`：防爆炸
- **词嵌入** `nn.Embedding`：把离散 ID 变成可学的向量

一句话总结

RNN 用“循环”把任意长度的序列压缩成一个固定大小的向量。简单优雅，但有根本性局限——信息瓶颈和无法并行。

下一讲（第 9 讲）：注意力机制与 Transformer 架构

- 编码器-解码器架构与“信息瓶颈”
- 注意力机制的直觉与公式
- Self-Attention 与 Q/K/V
- 多头注意力与完整 Transformer

课后练习

1. 用 `nn.GRU` 在 IMDB 影评数据上做二分类（正面/负面），目标精度 $\geq 85\%$
2. 比较 `nn.RNN` 和 `nn.GRU` 在同任务上的差异
3. 推导 BPTT 在 $T = 3$ 时的完整反向梯度公式
4. 思考题：为什么用 `tanh` 而不是 `ReLU` 作为 RNN 激活?

谢谢！

问题与讨论

下次课：注意力机制与 Transformer

孟翰　人工智能学院